

Metaheuristics for the Selective Capacitated Fair Pickup and Delivery Problem

Tina Taheri, Lorena Lozano

November 30, 2025

1 Introduction

The Selective Capacitated Fair Pickup and Delivery Problem (SCF-PDP) represents a complex optimization challenge that generalizes classic vehicle routing problems. Unlike standard VRPs, this problem requires selecting a subset of customer requests to serve from a pool of potential customers, ensuring that at least a minimum number (γ) are covered. Each request involves transporting goods from a specific pickup location to a drop-off location using a homogeneous fleet of vehicles with limited capacity (C).

The problem is modeled on a complete directed graph $G = (V, A)$, where the objective is not merely to minimize the total travel distance or duration, but to find a balance between efficiency and equity among the drivers. This is achieved through a composite objective function that minimizes the sum of total travel durations while simultaneously penalizing workload imbalances using a fairness weighting factor (ρ).

In this report, we present the design, implementation, and analysis of various heuristic and metaheuristic approaches to solve the SCF-PDP efficiently. Our study begins with deterministic and randomized constructive heuristics to generate initial feasible solutions. Subsequently, we explore Local Search methods, implementing four distinct neighborhood structures and efficient delta-evaluation mechanisms. Finally, we investigate advanced metaheuristics, including Variable Neighborhood Descent (VND), GRASP, and Beam Search, to escape local optima and ensure robust performance across diverse instance sizes.

2 Problem Representation

The SCF-PDP framework provides a reusable structure for representing and processing problem instances of the Static Capacitated Fair Pickup and Delivery Problem (SCF-PDP). Each instance file contains all required information about requests, vehicle constraints, and geometric locations. The Python implementation exposes these components through the `SCFPDPInstance` class.

2.1 Instance Structure

An instance begins with a header line containing five parameters:

$$(n, K, C, \gamma, \rho),$$

where

- n is the number of pickup-delivery requests,
- K is the number of available vehicles,
- C is the vehicle capacity,
- γ is the minimum number of requests that must be served,
- ρ is a fairness weight used in the objective function.

Each request i has an associated demand d_i , followed by two geometric locations: a pickup point p_i and a dropoff point q_i . The depot is denoted by location 0.

All coordinates are parsed from the instance file and stored as

$$\text{depot}, \quad \text{pickups}[1..n], \quad \text{dropoffs}[1..n].$$

2.2 Matrix-Based Distance Computation

To efficiently compute all pairwise distances between locations, the framework constructs a dense matrix

$$D \in \mathbb{R}^{(2n+1) \times (2n+1)}, \quad D_{ij} = \lceil \|x_i - x_j\|_2 \rceil,$$

where each x_i is a 2D coordinate and $\lceil \cdot \rceil$ is the ceiling operator.

The distance matrix computation uses PyTorch tensors to exploit automatic vectorization. Let X be the matrix of all coordinates stacked row-wise. PyTorch constructs the tensor

$$\Delta = X[:, \text{None}, :] - X[\text{None}, :, :],$$

which represents all pairwise differences. The Euclidean distances are then computed via

$$D = \left\lceil \sqrt{(\Delta^2) \mathbf{1}} \right\rceil,$$

where the summation over the last dimension reduces the coordinate difference to a scalar distance.

This results in a fully materialized integer distance matrix stored as `self.dist`, which is used throughout the rest of the framework for route evaluation and feasibility checking.

3 Solution Representation

A solution consists of K vehicle routes. Each route is represented as an ordered list of location indices:

$$R_k = (s_1, s_2, \dots, s_m), \quad s_j \in \{0, 1, \dots, 2n\}.$$

A route implicitly begins and ends at the depot. The framework provides tools for request extraction, route duration evaluation, vehicle load computation, and feasibility checking.

3.1 Route Feasibility

A route is feasible if it satisfies:

1. **Capacity constraints:** the load never exceeds C ,

$$\ell_j = \sum_{i: p_i \in R_k \text{ before } s_j} d_i - \sum_{i: q_i \in R_k \text{ before } s_j} d_i \Rightarrow 0 \leq \ell_j \leq C.$$

2. **Pickup-before-dropoff:** for each request i ,

$$\text{pos}(p_i) < \text{pos}(q_i).$$

3. **No duplication:** each request is served by at most one vehicle.

Additionally, the global minimum coverage constraint requires that at least γ distinct requests are served.

3.2 Route Duration

The duration of a route R_k is computed using the distance matrix:

$$\text{dur}(R_k) = D_{0,s_1} + \sum_{j=1}^{m-1} D_{s_j,s_{j+1}} + D_{s_m,0}.$$

3.3 Objective Function

The SCF-PDP objective balances total travel time and fairness across vehicles. For route durations $d_1, \dots, d_K$ the Jain fairness index is

$$J(d_1, \dots, d_K) = \frac{(\sum_k d_k)^2}{K \sum_k d_k^2}.$$

The full objective is

$$\text{Obj} = \sum_{k=1}^K d_k + \rho (1 - J(d_1, \dots, d_K)).$$

This encourages low total travel cost while penalizing imbalanced workloads across vehicles.

4 Implemented Algorithms

4.1 Construction Heuristics

4.1.1 Deterministic Construction Heuristic

The constructive phase builds an initial feasible solution by inserting requests one by one into the routes. The procedure is deterministic and greedy: in each step it selects the request and the route that minimizes the increase in the objective value.

Request Prioritization Each request r is assigned a priority score based on the direct round-trip distance of serving it alone:

$$d_r = d(0, p_r) + d(p_r, q_r) + d(q_r, 0),$$

where p_r and q_r denote the pickup and dropoff locations of the request. To avoid always preferring small requests, the distance is normalized by demand w_r :

$$\pi_r = \frac{d_r}{w_r + 1}.$$

Requests are processed in increasing order of π_r .

Route Insertion and Feasibility When evaluating a request r for insertion into a route k , we always append the sequence $[p_r, q_r]$ at the end of the current route. This simplifies load tracking, since the route load always returns to 0 after serving a full request. A route is feasible for insertion if the maximum route load after the insertion,

$$\max\{\text{current max load of route } k, w_r\},$$

does not exceed the vehicle capacity C .

Efficient Duration Update Let D_k be the current duration of route k . If the route is empty, the new duration is simply d_r . Otherwise, if the last stop of the route is ℓ , inserting r modifies the duration by removing the final return-to-depot leg and adding the new travel arcs:

$$D'_k = D_k - d(\ell, 0) + d(\ell, p_r) + d(p_r, q_r) + d(q_r, 0).$$

Delta Evaluation of the Objective The global objective function is

$$F = \sum_k D_k + \rho(1 - J),$$

where J is the Jain fairness index computed over the non-empty routes:

$$J = \frac{(\sum_k D_k)^2}{m \sum_k D_k^2},$$

and m is the number of active routes. For each candidate route k , we compute the updated terms:

$$\sum_k D'_k, \quad \sum_k (D'_k)^2, \quad m',$$

and evaluate the updated fairness J' . The change in the objective is then

$$\Delta F_k = \left(\sum_k D'_k - \sum_k D_k \right) + \rho(J - J').$$

The request is assigned to the route k that minimizes ΔF_k .

Updating Global Quantities After selecting the best route, all global statistics are updated:

$$\sum_k D_k, \quad \sum_k D_k^2, \quad m, \quad J, \quad F.$$

The algorithm continues until either all requests are processed or the required minimum number of served requests γ is reached.

4.1.2 Randomized Construction Heuristic

The randomized variant follows the same insertion logic, feasibility rules, and delta-based objective evaluation as the deterministic greedy heuristic. However, instead of always selecting the route that yields the minimum increase in the objective, it introduces controlled randomness. Only the new components are described below.

Randomness Parameter. A parameter $\alpha \in [0, 1]$ controls the degree of randomness:

$$\alpha = 0 \Rightarrow \text{pure greedy}, \quad \alpha = 1 \Rightarrow \text{uniform random among feasible routes}.$$

Intermediate values generate a *biased randomized choice*.

Candidate Set. For each request r , all feasible insertions are evaluated and collected into a candidate list

$$\mathcal{C} = \{(k, \Delta F_k) \mid \text{route } k \text{ feasible for } r\},$$

where ΔF_k is the change in the objective if r is appended to route k . This list is sorted in ascending order of ΔF_k .

Probabilistic Route Selection. Let the sorted candidates be indexed as

$$(k_0, \Delta_0), (k_1, \Delta_1), (k_2, \Delta_2), \dots,$$

where Δ_0 is the smallest (best) increase in the objective. A weight is assigned to each candidate according to its rank:

$$w_i = (1 - \alpha)^i.$$

Thus, low-rank (greedy) candidates receive higher weight, while higher-rank candidates remain selectable but less likely.

The probability of selecting candidate i is

$$P(i) = \frac{w_i}{\sum_j w_j}.$$

A route is then drawn according to this distribution. This implements a GRASP-style randomized restricted candidate list (RCL).

4.1.3 Beam Search

The beam search construction heuristic maintains a set of w partial solutions (the "beam") at each step, where w is the beam width parameter. At each iteration, the algorithm expands all partial solutions in the beam by attempting to insert the next request, then retains only the w best solutions based on their objective values.

Algorithm Overview The algorithm proceeds as follows:

1. **Request Prioritization:** Requests are sorted by priority $p_i = \frac{d_{0,p_i} + d_{p_i,d_i} + d_{d_i,0}}{q_i + 1}$, where p_i and d_i are the pickup and dropoff locations of request i , and q_i is the demand. This prioritizes requests with lower cost-to-demand ratios.
2. **Beam Initialization:** The beam is initialized with a single empty partial solution containing no assigned requests.
3. **Request Insertion:** For each request in priority order, the algorithm:
 - Expands each partial solution in the current beam by attempting to insert the request into each feasible vehicle
 - For each vehicle, finds the best insertion position that minimizes the incremental cost while maintaining capacity and precedence constraints
 - Creates a new partial solution for each feasible insertion
 - Sorts all generated partial solutions by objective value and retains only the top w solutions for the next iteration
4. **Termination:** The algorithm terminates when all requests have been considered or when all partial solutions in the beam have served exactly γ requests.

Key Optimization Techniques **Delta-based Representation:** Instead of maintaining complete solution copies, each partial solution stores only the delta (modification) from its parent solution. This includes the vehicle index, request, pickup/dropoff locations, and incremental cost. The full solution is reconstructed only at the end by traversing the parent chain, reducing memory usage and copy overhead from $O(n)$ to $O(1)$ per expansion.

Adaptive Beam Width: The beam width is automatically adjusted based on instance characteristics:

$$w = \begin{cases} 1 & \text{if } n \geq 8000 \\ 2 & \text{if } n \geq 5000 \\ 3 & \text{if } n \geq 2000 \text{ or } \gamma/n > 0.85 \\ 5 & \text{otherwise} \end{cases} \quad (1)$$

This adaptive strategy balances solution quality and computational efficiency, using narrower beams for larger instances to manage exponential growth in computational requirements.

Request Pruning: For large instances with low γ/n ratios (below 0.85), the algorithm only considers the $\min(n, \max(1.3\gamma, \gamma + 1000))$ highest-priority requests rather than all n requests. This exploits the observation that low-priority requests are unlikely to appear in optimal solutions when $\gamma \ll n$.

Route Simplification: For routes with more than 50 stops, the algorithm only attempts to append new requests at the end of the route rather than exploring all possible insertion positions. This heuristic trades solution quality for computational speed on complex routes, as finding optimal insertion positions becomes prohibitively expensive.

Early Termination: The algorithm terminates immediately when all partial solutions in the beam have reached exactly γ served requests, avoiding unnecessary computation.

Complexity Analysis In the worst case, the algorithm has time complexity $O(n \cdot w \cdot K \cdot s^2)$, where n is the number of requests, w is the beam width, K is the number of vehicles, and s is the average number of stops per route. The s^2 factor comes from finding the best insertion position (trying all pairs of positions for pickup and dropoff). However, the adaptive beam width, request pruning, and route simplification techniques significantly reduce the effective complexity for large instances.

4.2 Neighborhood Structures

To effectively traverse the solution space of the SCF-PDP, we implemented three distinct neighborhood structures. These neighborhoods range from simple modifications within a single route to complex exchanges between routes and the pool of unserved requests.

For all neighborhoods, we employ a Delta Evaluation strategy. Instead of recalculating the objective function from scratch (which requires iterating over all vehicles), we strictly calculate the change in duration for the modified routes and update the global statistics (`sum_durations`, `sum_sq_durations`) in constant time.

4.2.1 Neighborhood 1: Intra-Route Relocation (N1)

This neighborhood focuses on optimizing the sequence of stops within a single route without altering the assignment of requests to vehicles.

Given a solution S , the algorithm iterates through every served request r . For the route k serving r , the request (both pickup and drop-off nodes) is temporarily removed. We then attempt to re-insert the pair into all possible positions (i, j) of the current route sequence, subject to the constraint $i < j$.

Since the modification is local to a single route k , we only recalculate the duration D_k . The change in the global objective is derived by subtracting the old contribution of D_k and adding the new one. Therefore for a route with length L , there are $O(L^2)$ insertion positions. With n total served requests, the size of this neighborhood is approximately $O(nL^2)$.

4.2.2 Neighborhood 2: Inter-Route Relocation (N2)

This structure allows moving a customer request from its current route to a different vehicle.

The algorithm selects a request r currently served by vehicle k_{source} and attempts to move it to every other vehicle k_{dest} . The pair of nodes is inserted into k_{dest} at the position that minimizes the increase in duration, provided the vehicle has sufficient remaining capacity. Two routes are modified: k_{source} (shortened) and k_{dest} (lengthened). We update the global sums by removing the old durations of both vehicles and adding their new durations. For this reason we iterate over n requests and try to move them to $K - 1$ other vehicles. For each target vehicle, we check $O(L^2)$ positions. Thereby the complexity is roughly $O(n \cdot K \cdot L^2)$.

4.2.3 Neighborhood 3: Swap Requests (N3)

When vehicles are operating near full capacity, simple relocation (N2) often fails due to capacity constraints. The Swap neighborhood addresses this by exchanging two requests between different routes.

We select two served requests, r_1 from route k_1 and r_2 from route k_2 . The operation swaps their assignments: r_1 is moved to k_2 and r_2 is moved to k_1 . We search for the best insertion positions for the new requests in their respective new routes.

Similar to N2, only two routes are modified. The cost calculation is done incrementally by evaluating the duration changes in k_1 and k_2 independently before updating the global objective. There are $O(n^2)$ pairs of requests. For each swap, we perform insertion checks in two routes, leading to a complexity of $O(n^2 L^2)$.

4.3 Local Search Framework

We developed a modular Local Search engine capable of accepting any solution, neighborhood structure, and evaluation strategy. This flexibility allows us to reuse the same core logic for simple iterative improvement as well as within more complex metaheuristics like GRASP and VND.

The general procedure takes an initial solution S and iteratively replaces it with a neighbor $S' \in N(S)$ such that $f(S') < f(S)$. The search terminates when no improving neighbor

is found (local optimum) or when a termination criterion (time limit or maximum iterations) is met.

To ensure a fair comparison across all variants, all local search experiments presented in this section were initialized using the Deterministic Construction Heuristic (described in Sec. 4.1.1).

4.3.1 Step Functions

Our framework supports two distinct strategies for selecting the next move, offering a trade-off between computation speed and convergence quality:

- **Best Improvement:** The algorithm evaluates the entire neighborhood $N(S)$ (or a significant subset for very large instances) and selects the move with the most negative Δ (greatest improvement).
- **First Improvement:** The algorithm scans the neighborhood and accepts the first neighbor found that improves the objective function $f(S') < f(S)$. This strategy allows for very fast iterations, preventing the search from spending too much time evaluating moves in massive neighborhoods.

To handle the strict runtime limits (15 minutes), we incorporated a sampling parameter `max_neighbors_per_iter`. For extremely large neighborhoods where a full scan is prohibitive, the Best Improvement strategy evaluates only a fixed number of random candidates, effectively turning into a partial best improvement strategy.

4.3.2 Variable Neighborhood Descent (VND)

To escape local optima encountered by simple local search, we implemented a Variable Neighborhood Descent (VND) framework.

Our implementation is designed to handle the wide range of instance sizes. To ensure the algorithm stays within the time limit while maintaining search effectiveness, we implemented an adaptive sampling strategy: For small instances ($n < 500$), the local search explores the entire neighborhood. For large instances ($n \geq 1000$), the algorithm limits the search to a random subset of neighbors per iteration. This allows the VND to complete cycles even on massive datasets.

The VND starts with a solution generated by the Deterministic Greedy Construction described in Sec 4.1.1). It iterates through a set of neighborhood structures $N_k, k = 1, \dots, k_{max}$. The algorithm applies Local Search in N_k to reach a local optimum. If an improvement is found, the search centers on the new solution, and the neighborhood index is reset to the first one. If no improvement is found, the algorithm proceeds to the next structure.

To analyze the impact of neighborhood ordering and step functions, we developed four distinct VND configurations:

- **VND Standard Best:** Uses the sequence $N_1 \rightarrow N_2 \rightarrow N_3$ with a Best Improvement strategy. This orders neighborhoods by increasing complexity and computational cost.
- **VND Standard First:** Uses the same sequence ($N_1 \rightarrow N_2 \rightarrow N_3$) but employs First Improvement. This variant is designed to test if faster iterations yield better results than exhaustive search given the time constraints.

- VND Reverse: Uses the sequence $N_3 \rightarrow N_2 \rightarrow N_1$ (First Improvement). This configuration tests the hypothesis that performing heavy structural changes before fine-tuning might allow the algorithm to navigate the solution space more broadly before settling into local optima.
- VND Light: Uses only $N_1 \rightarrow N_2$ (omitting N_3). Since the Swap neighborhood (N_3) is computationally expensive, this variant acts as a study to determine if the marginal gain from N_3 justifies its runtime cost compared to running more iterations of lighter neighborhoods.

4.4 Metaheuristics

4.4.1 GRASP

We implemented a GRASP metaheuristic. This iterative process consists of two main phases repeated until a termination criterion is met.

Our implementation is specifically designed to maximize performance within the strict runtime limit of the competition.

- Construction Phase: In each iteration, a solution is generated using the Randomized Greedy Heuristic (described in Sec 4.1.2) with a randomization factor $\alpha = 0.3$. To ensure diversity, each iteration utilizes a different random seed.
- Local Search Phase: The constructed solution is immediately improved using our Local Search framework. The specific neighborhood structure and step function used were determined through experimental tuning (see below).
- Adaptive Time Management: Given the variation in instance sizes, a fixed number of iterations is infeasible. We implemented a dynamic iteration target: small instances ($n < 500$) target 50 iterations, medium instances ($n < 2000$) target 30 iterations and very large instances ($n \geq 5000$) target 5 iterations. Additionally, a predictive termination check is performed after each iteration. If the remaining time is less than the average duration of the previous iterations (plus a safety buffer of 30 seconds), the algorithm terminates early to guarantee a valid output file.

4.4.2 Tabu Search

Tabu search is a local search metaheuristic that iteratively explores the solution space while avoiding cycles and previously visited solutions.

Overview. Given an initial solution S_0 using randomized greedy heuristic, the algorithm explores a set of *neighborhoods* to iteratively improve the solution. At each iteration, the best or first step function improving neighbor is selected, and certain moves are temporarily forbidden using a *tabu list* to prevent cycling.

Neighborhoods. The tabu search considers the following neighborhoods N_1 , N_2 , N_3 as described before.

Tabu List. The tabu list stores recently applied moves for a tenure of T iterations. Moves that are tabu cannot be reversed unless an *aspiration criterion* is met (e.g., the move improves the global best solution).

Dynamic Parameters and Time Management. Tabu search dynamically adapts parameters based on instance size and remaining time:

- **Tabu tenure T :** shorter for very large instances to allow faster exploration.
- **Maximum neighbors per iteration:** reduced as time pressure increases.
- **Maximum iterations:** estimated based on remaining time and average iteration duration.

Algorithm Flow.

1. Construct initial solution using a chosen heuristic.
2. Initialize tabu list and counters for iterations and improvements.
3. While iteration limit or time limit not reached:
 - Generate feasible neighbors from all defined neighborhoods.
 - Apply tabu restrictions and aspiration criteria.
 - Select neighbor according to the chosen step function.
 - Update current solution and tabu list.
 - Update global best if improvement occurs.
 - Adapt exploration parameters dynamically based on convergence and time pressure.
4. Return the best solution found.

5 Parameter Tuning

5.0.1 Randomized Deterministic Heuristic

Experimental Setup We tuned the randomization parameter α of the randomized greedy construction heuristic, where $\alpha = 0$ represents pure greedy behavior and $\alpha = 1$ represents uniform random selection. We tested $\alpha \in \{0.0, 0.1, 0.2, 0.3, 0.4, 0.5, 0.7, 1.0\}$ on 35 instances spanning problem sizes from 50 to 10,000 requests.

Results Table 1 presents the performance of all tested configurations.

Table 1: Randomized greedy parameter configurations

α	Avg Objective	Std Dev	Avg Time (s)
0.2	460,858	776,413	0.618
0.1	483,696	805,337	0.309
0.5	506,362	879,830	0.537
0.3	506,753	877,803	0.309
0.4	507,151	880,252	0.392
0.7	517,103	900,003	0.263
0.0	641,073	1,120,543	0.313
1.0	641,073	1,120,543	0.306

Analysis The α parameter shows **high sensitivity** (33.81% variation), with an objective range of 180,215 across configurations. Key findings:

- **Optimal value:** $\alpha = 0.2$ achieves the best solution quality (460,858), suggesting that moderate randomization helps escape local optima while maintaining greedy guidance.
- **Extreme values fail:** Both $\alpha = 0.0$ (pure greedy) and $\alpha = 1.0$ (pure random) produce the worst results (641,073), demonstrating the importance of balancing exploration and exploitation.
- **Quality-speed trade-off:** The fastest configuration ($\alpha = 0.7$, 0.263s) sacrifices 12% solution quality compared to optimal. For balanced performance, $\alpha = 0.1$ offers good quality (483,696) with fast runtime (0.309s).

Validation on Test Instances We validated the best configuration ($\alpha = 0.2$) on unseen test instances. The test instances with $\alpha = 0.2$ achieve comparable performance to training with $\alpha = 0.1$, with performance gaps ranging from -4.9% to +6.7% across problem sizes. All 15 test instances maintained 100% feasibility, validating robust generalization. The configuration demonstrates excellent scalability, solving 10,000-request instances in under 27 seconds. These results confirm that moderate randomization provides consistent performance across diverse problem instances without overfitting to training data.

5.0.2 Local Search

Experimental Analysis and Configuration: To determine the most effective configuration, we evaluated the performance of the implemented neighborhood structures (N_1 , N_2 , and N_3) combined with both step functions across all instance sizes. The results yield the following conclusions:

1. **Impact of Instance Size on Neighborhood Performance:** We observed a distinct crossover point in performance based on the problem scale.
 - **Small Instances ($n = 50$):** Neighborhood 1 (Intra-Route Relocation) achieved the best objective values. In these cases, the construction heuristic provides a reasonable request assignment, and the primary gain comes from optimizing the visiting sequence within each route (TSP-like optimization).

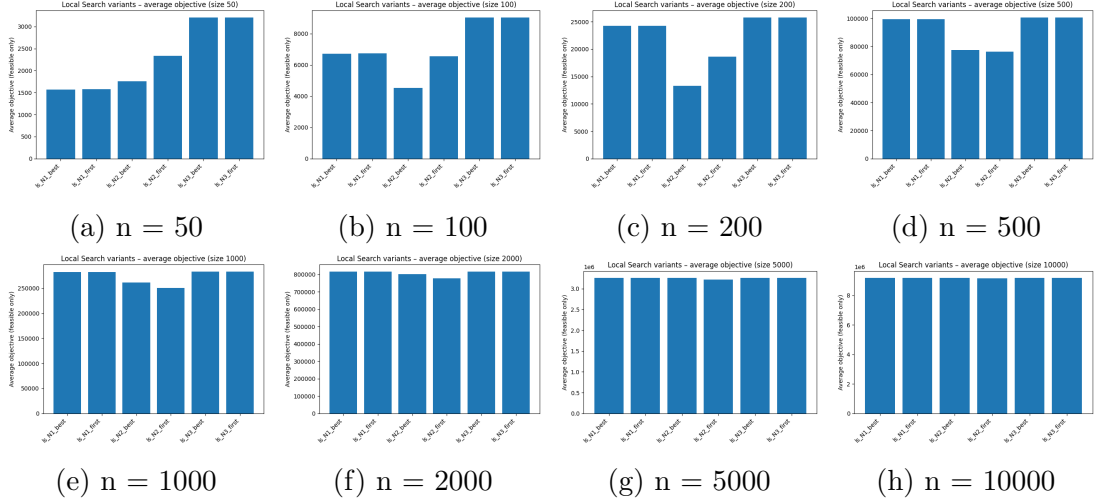


Figure 1: Local Search average objective function values

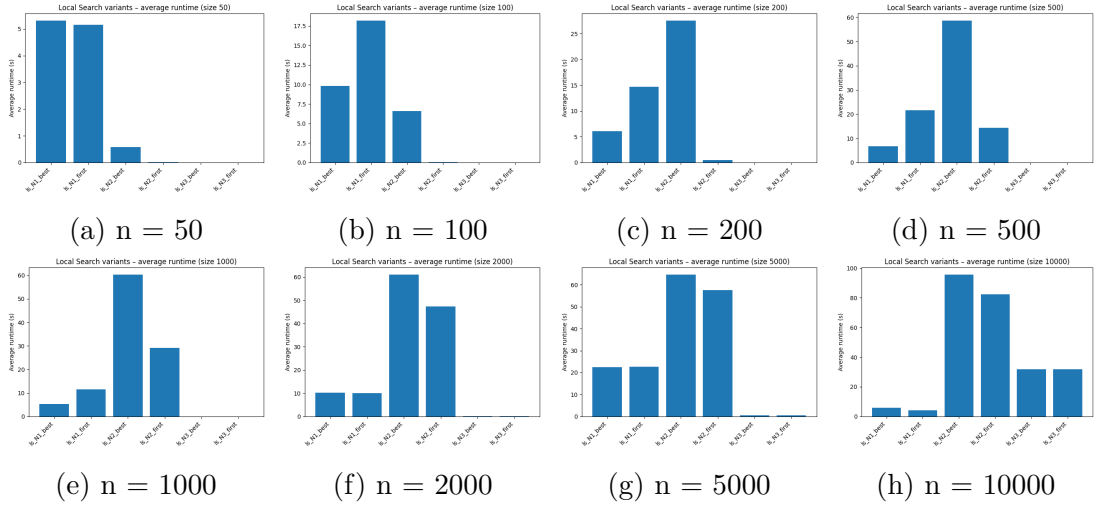


Figure 2: Local Search average runtime

- **Medium and Large Instances ($n \geq 100$):** Neighborhood 2 (Inter-Route Relocation) consistently outperformed N_1 . As the number of requests grows, the initial greedy assignment tends to produce unbalanced routes (negatively impacting the fairness metric ρ). N_2 becomes crucial here as it allows rebalancing the workload between vehicles, which provides a larger reduction in the global objective than merely optimizing the sequence of individual routes.

2. Scalability of Step Functions in N_2 : Analyzing the Inter-Route neighborhood (N_2) specifically, we found a trade-off between exploration depth and runtime:

- For **lower** n , the *Best Improvement* strategy yielded slightly better solutions, as it could exhaustively search for the optimal move.
- For **higher** n , the *First Improvement* strategy proved superior. The search space for inter-route moves grows as $O(n \cdot K \cdot L^2)$. *Best Improvement* becomes computationally prohibitive, while *First Improvement* allows the algorithm to

perform many more moves within the time limit, effectively navigating the search space better on large-scale problems.

Selected Configuration: Based on this analysis, we selected Neighborhood 2 (Inter-Route) with First Improvement as our primary standalone Local Search configuration for general application.

5.0.3 Variable Neighborhood Descendent (VND)

Performance Comparison : We compared the four configurations across the full range of instance sizes.

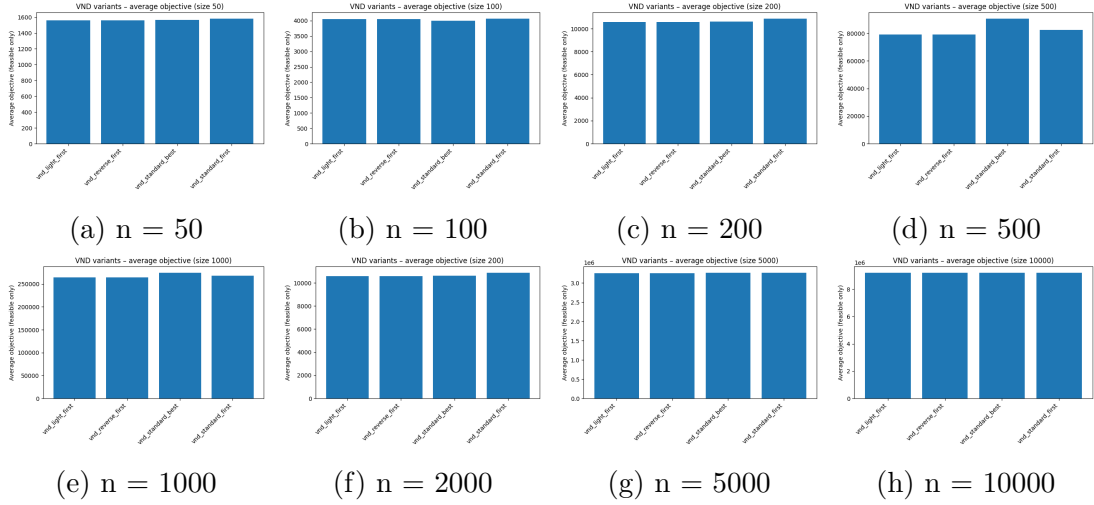


Figure 3: VND average objective function values

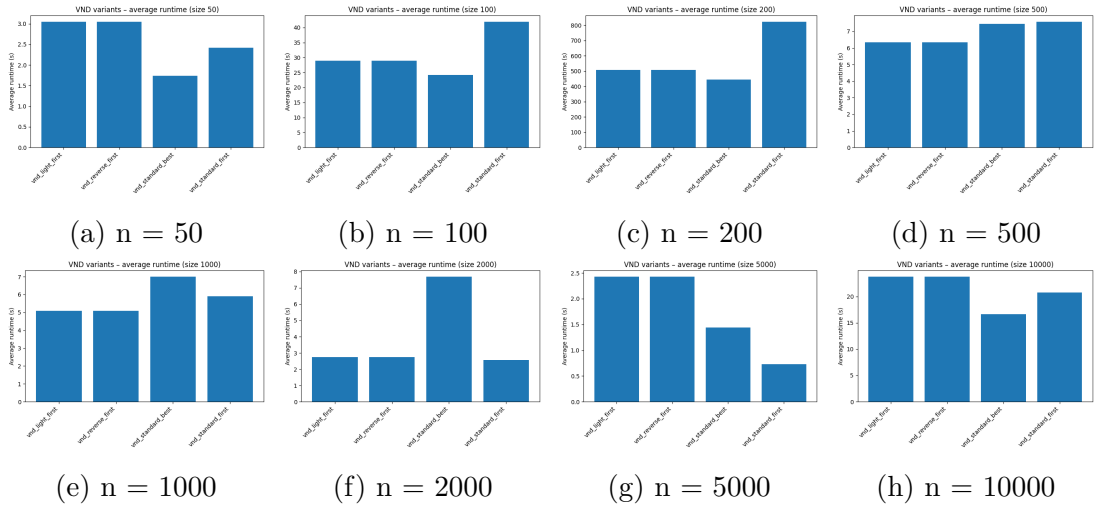


Figure 4: VND average runtime

Regarding **runtime**, the *VND Standard Best* configuration proved to be the most efficient overall. It consistently achieved the lowest average runtime on small instances (sizes 50, 100, 200) and remained highly competitive on the largest instances (size 10000),

outperforming the First Improvement variants in many cases except in the case of medium sizes.

Regarding the **objective function value**, the behavior varies according to the instance size:

- **Small sizes:** The *VND Light*, *VND Reverse*, and *VND Standard Best* variants perform slightly better than *VND Standard First*, achieving lower objective values.
- **Medium sizes:** We observed a shift where *VND Standard Best* performs slightly worse than the other three variants.
- **Large sizes:** As the complexity increases, the distinction between strategies blurs. For the largest instances, all four configurations converge to similar objective function values, suggesting that the sampling strategy and the vast search space become the dominant factors over the specific neighborhood ordering or pivot rule.

Selected Configuration for Final Comparison: Based on the analysis, we selected VND Standard Best as the best-performing configuration. Although it shows a slight deviation in solution quality for medium-sized instances, it demonstrates superior computational efficiency across the majority of test cases—particularly in small and very large instances—while maintaining competitive solution quality.

5.0.4 GRASP

Experimental Tuning and Variant Selection: To determine the most effective configuration for the Local Search phase within GRASP, we conducted a preliminary study on the training instances of size $n = 50$. We evaluated 6 variants combining our three active neighborhood structures (N_1 to N_3) with both *Best Improvement* and *First Improvement* strategies. Our analysis of the results yielded the following insights:

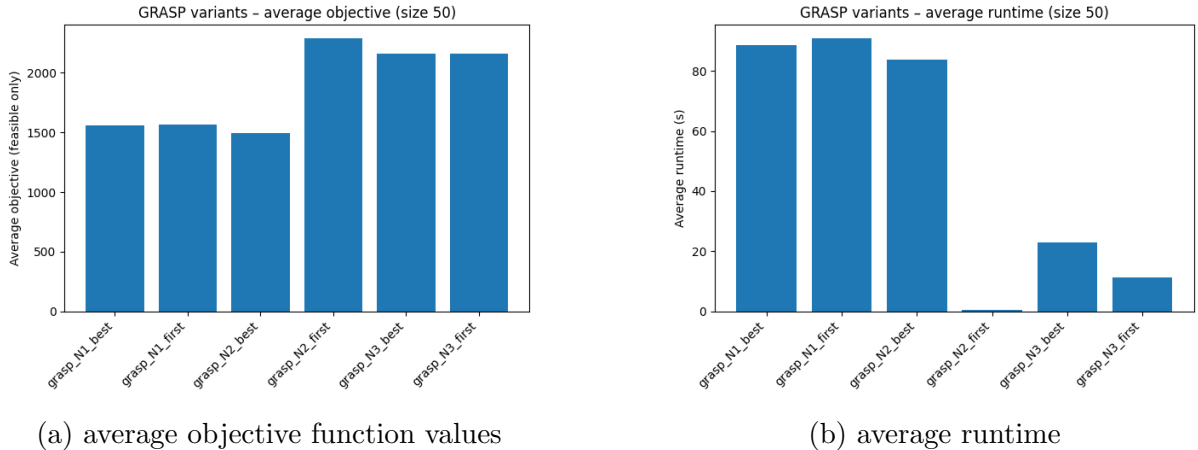


Figure 5: GRASP experiment results (n=50)

- **Neighborhood Dominance (N_1):** The variants utilizing Neighborhood 1 (Intra-Route) achieved the lowest (best) objective values on these small instances. This aligns with our Local Search findings where intra-route optimization effectively acts as a TSP solver, which is the dominant factor for small datasets.

- **Performance Anomaly in N_2 (First Improvement):** We observed that the `grasp_N2_first` variant performed poorly on these small instances, exhibiting an unusually high objective value and near-zero runtime. This indicates that for small n , the *First Improvement* scan of the Inter-Route neighborhood often fails to find any improving move immediately after the randomized construction, resulting in premature stagnation. In contrast, `grasp_N2_best` managed to find improvements but at a higher computational cost.
- **Inefficiency of Swap (N_3):** As seen in previous sections, the Swap neighborhood proved to be computationally expensive without delivering competitive objective values compared to N_1 .

Selected Configuration for Final Comparison: Based on the experimental results from the tuning set ($n = 50$) and the scalability analysis, we selected GRASP with N_1 (Intra-Route) using First Improvement as our final configuration. This decision rests on two key factors:

1. **Objective Quality:** The variants utilizing N_1 consistently achieved the lowest objective values during the tuning phase, proving that optimizing the visiting sequence is the most impactful factor for these instances.
2. **Reliability and Speed:** Unlike N_2 , which showed signs of premature stagnation on small instances, N_1 provided robust results. Furthermore, the choice of *First Improvement* minimizes the time spent in the Local Search phase. This is strategic for the GRASP metaheuristic: a faster local search allows the framework to perform a higher number of global iterations (construction + improvement) within the time limit, increasing the probability of escaping local optima.

5.0.5 Beam Search Parameter Tuning

Experimental Setup To determine the optimal beam width parameter, we conducted tuning experiments on instance sizes 50, 100, 200, 500, and 1000. Larger instances (2000, 5000, 10000) were excluded from the tuning phase due to their prohibitive computational requirements, as beam search runtime grows exponentially with both instance size and beam width. We tested three beam width configurations: 2, 3, and 5. Each configuration was evaluated on 5 instances per size to balance statistical reliability with computational efficiency, resulting in a total of 75 experimental runs (5 sizes $\times$ 3 configurations $\times$ 5 instances).

Results Table 2 presents the average objective values and runtimes for each combination of instance size and beam width. The results demonstrate consistent patterns across all instance sizes: beam width 5 achieves the best solution quality, while beam width 2 provides the fastest computation times. Beam width 3 shows performance nearly identical to beam width 2 for smaller instances (50-200), but begins to show improvement for larger instances (500-1000).

Table 2: Beam search parameter tuning: average performance by instance size and beam width

Size	Beam Width	Avg. Objective	Avg. Runtime (s)
50	2	3,345.07	1.697
50	3	3,345.07	1.697
50	5	3,289.20	2.990
100	2	8,897.03	2.079
100	3	8,897.03	2.091
100	5	8,773.03	4.186
200	2	23,230.14	3.508
200	3	23,230.14	3.512
200	5	22,882.52	8.378
500	2	84,756.27	5.115
500	3	84,756.27	5.181
500	5	83,839.46	11.992
1000	2	236,521.70	11.777
1000	3	231,761.16	13.608
1000	5	223,018.17	26.543

Analysis The tuning experiments reveal a clear trade-off between solution quality and computational time. Increasing beam width from 2 to 5 consistently improves solution quality across all instance sizes, with improvements ranging from 1-3% for smaller instances (50-200 requests) to 6-8% for larger instances (500-1000 requests). However, this improvement comes at a significant computational cost: beam width 5 requires approximately 80-100% more runtime than width 2. For instance size 1000, beam width 5 achieves an average objective of 223,018 compared to 236,522 for width 2 (a 5.7% improvement), but runtime increases from approximately 13 seconds to 26 seconds.

Beam width 3 offers minimal benefit over width 2 while adding runtime overhead. For smaller instances (50-200), the objective values are identical between widths 2 and 3, while for larger instances the improvement is marginal (approximately 2% for size 1000). This suggests that beam width 3 does not provide sufficient exploration to escape local optima effectively.

Based on these results, we selected beam width 5 for the final experiments. While this configuration incurs higher computational costs, the consistent and substantial improvements in solution quality justify its use, particularly for larger instances where the quality gains are most pronounced. The increased runtime remains acceptable given the offline nature of the optimization problem and the value of higher-quality solutions.

Validation on Test Instances We evaluated beam width 5 on test instances of sizes 50, 100, 200, 500, and 1000. The results follow the same trends observed during tuning: solution quality increases with instance size, and runtime grows moderately. Compared to the training/tuning instances, test objectives are slightly higher for larger sizes (e.g., 242,340 vs. 223,018 for size 1000), reflecting instance-specific variability, but remain consistent with the expected performance. This confirms that the selected beam width maintains a strong balance of solution quality and computational efficiency on unseen data.

5.0.6 Tabu Searching

The experimental framework evaluates four distinct tabu tenure values: 5, 7, 10, and 15. These values represent a spectrum from aggressive (short tenure, allowing rapid revisitation of solutions) to conservative (long tenure, enforcing extended memory).

Table 3: Overall Performance Metrics by Configuration

Tabu Tenure	Avg Objective	Avg Runtime (s)	Avg Iterations	Instances
5	89,384.91	8.95	85.4	30
7	89,293.55	9.02	90.5	30
10	90,838.08	8.72	87.9	30
15	90,194.33	9.39	88.7	30

Note: Runtime includes construction phase (~ 0.05 - 0.10 s) and tabu search. Differences (8.72-9.39s) are within experimental variance.

Table 4: Average Objective Values for Small Instances

Size	Tabu=5	Tabu=7	Tabu=10	Tabu=15	Best
50	1,613.54	1,599.44	1,620.52	1,620.03	T=7
100	6,052.53	5,769.03	5,956.61	5,949.28	T=7
200	22,609.14	22,213.65	22,302.83	22,342.44	T=7
500	81,500.72	82,084.92	81,783.38	82,097.58	T=5
1000	211,809.74	212,852.72	221,157.51	215,031.78	T=5

The configuration with tabu tenure of 7 achieves the best average objective value at 89,293.55, representing a 0.1% improvement over tenure 5 and a 1.7% improvement over tenure 10. Also this superior performance comes with negligible computational overhead—the 9.02 second average runtime differs by less than 0.7 seconds from the fastest configuration, a difference within normal experimental variance.

All reported times include an initial construction phase lasting approximately 0.05-0.10 seconds, followed by the tabu search iterations. The small variations in total runtime (8.72 to 9.39 seconds) primarily reflect differences in iteration counts rather than fundamental algorithmic efficiency differences. The similar iteration counts across configurations (85.4 to 90.5 on average) suggest that convergence behavior is primarily driven by solution improvement opportunities rather than tabu list restrictions.

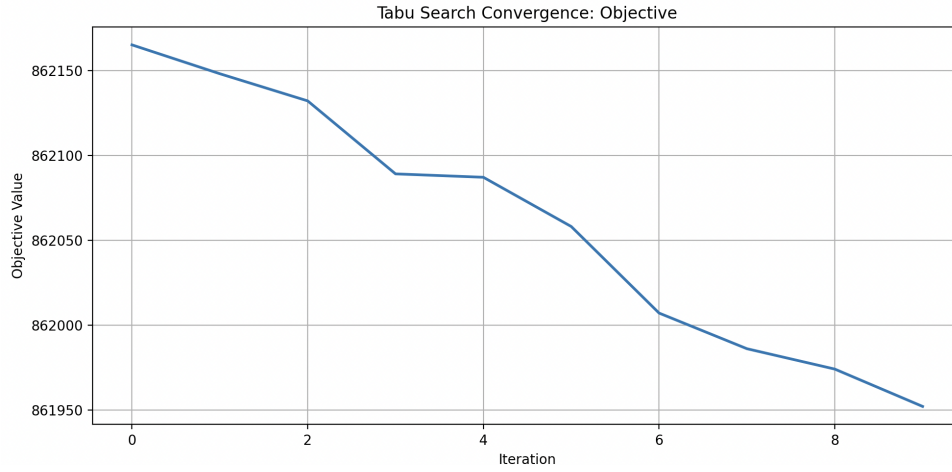


Figure 6: Convergence of the Tabu Search: evolution of objective value and fairness over iterations.

5.0.7 Parameter Settings

Algorithm	Parameter(s)	Value / Description
Deterministic Greedy	—	Inserts requests in priority order.
Randomized Greedy	α	0.2 (0 = greedy, 1 = random).
Beam Search	beam width	5 (partial solutions per step).
Local Search	Iters, Time, Neighbors	Max: 400 iters, 60s, 5000 neighbors.
GRASP	Iterations, Time	5–50 iters (adaptive), 870s limit.
VND	Sampling, Iters	50–800 neighbors, 1000 iters, 880s.
Tabu Search	step, tenure, neighbors	best/first, 3–15, 50–5000 (adaptive).

6 Computational Results

6.1 Greedy Construction Heuristic

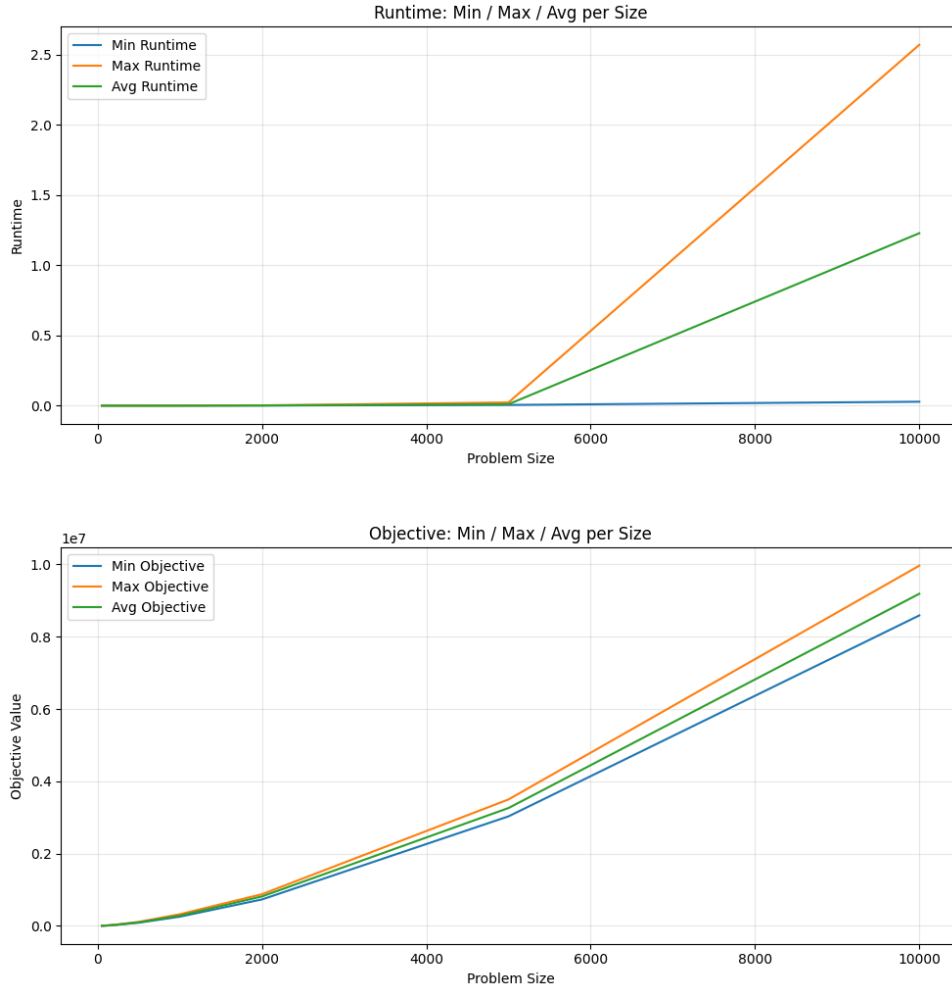


Figure 7: Runtime statistics (top) and objective function statistics (bottom): comparison of min/max/avg metrics.

Table 5: Performance summary of greedy construction heuristic by instance size

Size	Instances	Avg. Objective	Min Runtime	Max Runtime	Avg. Runtime
50	30	3,197.38	0.000	0.000	0.000
100	30	9,086.15	0.000	0.000	0.000
200	30	25,769.99	0.000	0.000	0.000
500	30	99,474.10	0.000	0.000	0.000
1,000	30	283,377.79	0.000	0.001	0.000
2,000	30	816,615.51	0.001	0.002	0.001
5,000	30	3,252,644.27	0.005	0.023	0.010
10,000	30	9,211,443.42	0.028	2.569	1.223

The greedy construction heuristic demonstrates excellent performance across all instance sizes. All 270 instances were solved to feasibility with very fast runtimes. For small

to medium instances (50-2000 requests), the algorithm runs in negligible time (≤ 0.002 s). Even for the largest instances with 10,000 requests, the average runtime remains under 1.3 seconds, with most instances solved in approximately 1 second. The objective values scale linearly with problem size, indicating consistent solution quality across different instance dimensions.

6.2 Randomized Greedy Construction

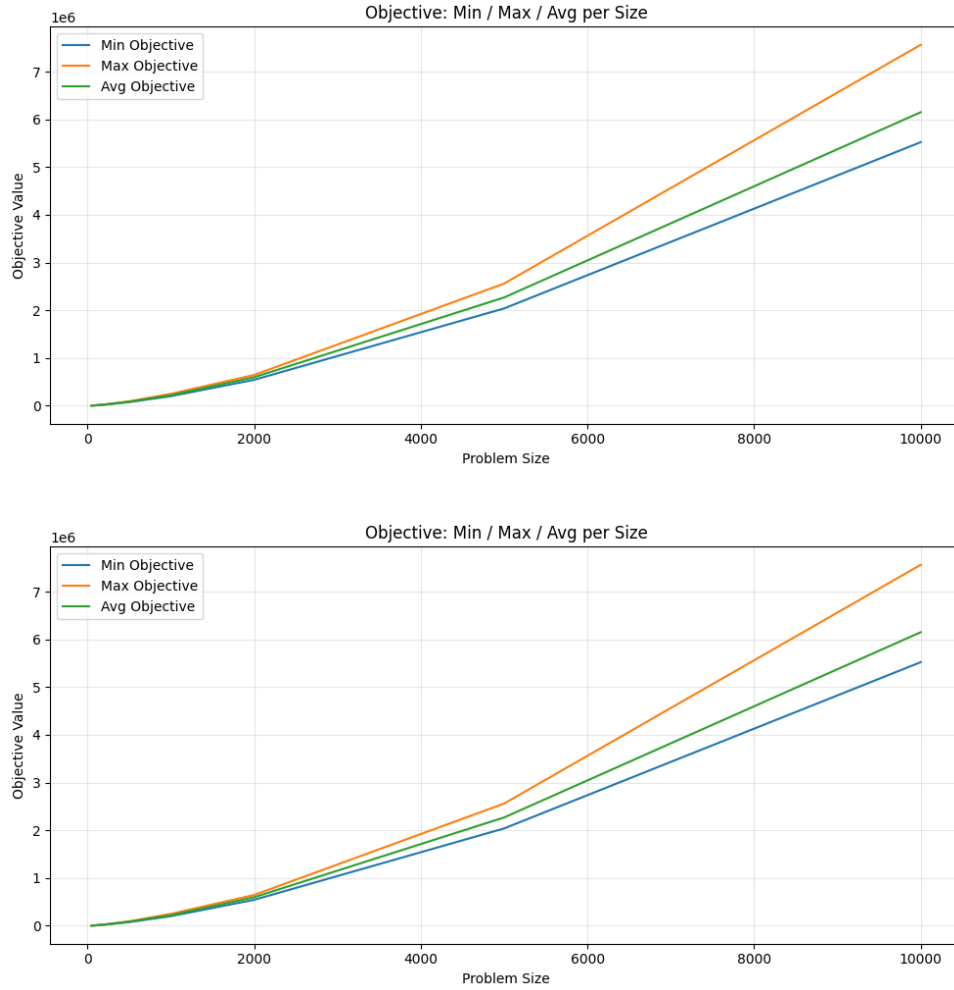


Figure 8: Runtime statistics (top) and objective function statistics (bottom): comparison of min/max/avg metrics.

Table 6: Performance summary of randomized construction heuristic ($\alpha = 0.2$) by instance size

Size	Instances	Avg. Objective	Min Runtime	Max Runtime	Avg. Runtime
50	30	3,089.31	0.000	0.000	0.000
100	30	8,840.01	0.000	0.000	0.000
200	30	24,320.44	0.000	0.000	0.000
500	30	84,493.26	0.000	0.000	0.000
1,000	30	220,975.91	0.000	0.002	0.001
2,000	30	596,145.44	0.001	0.002	0.002
5,000	30	2,269,664.69	0.009	0.024	0.014
10,000	30	6,137,040.95	0.043	0.712	0.176

6.3 Local Search

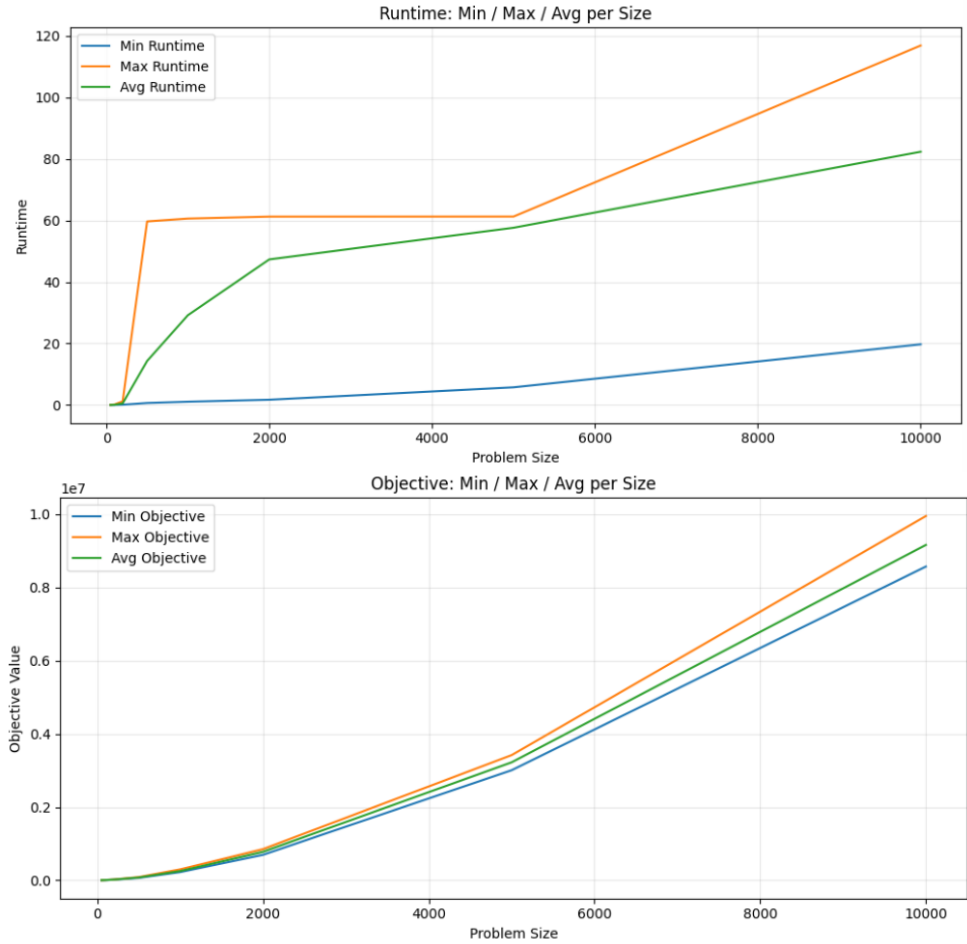


Figure 9: Runtime statistics (top) and objective function statistics (bottom): comparison of min/max/avg metrics.

Table 7: Performance summary based on aggregated statistics by instance size

Size	Instances	Avg. Objective	Min Runtime	Max Runtime	Avg. Runtime
50	30	2,331.51	0.003	0.032	0.011
100	30	6,551.14	0.013	0.159	0.067
200	30	18,604.95	0.083	1.190	0.510
500	30	76,293.07	0.640	59.665	14.303
1,000	30	250,586.50	1.055	60.587	29.169
2,000	30	778,735.60	1.701	61.225	47.322
5,000	30	3,222,376.00	5.745	61.245	57.620
10,000	30	9,163,011.00	19.719	116.852	82.333

The performance analysis of the Local Search algorithm, utilizing the N_2 (Swap) neighborhood structure with a *First Improvement* strategy, yields several insights as illustrated in the following figure and table.

The runtime statistics demonstrate that the N_2 First Improvement configuration is highly scalable. As shown, the algorithm maintains acceptable execution times even for the largest instances. This sub-quadratic growth in runtime confirms the efficiency of the *First Improvement* strategy. By accepting the first improving move found rather than scanning the entire $O(N^2)$ swap neighborhood, the algorithm avoids prohibitive computational costs on large datasets.

The objective function statistics reveal a linear growth trend relative to the problem size. A notable characteristic observed in the data is the low variance between the minimum, maximum, and average objective values across all sizes; the algorithm consistently converges to solutions of similar quality regardless of the specific instance topology or random initialization. However, while stable, the method is prone to getting trapped in local optima typical of simple Local Search. The fast convergence times suggest that the search quickly reaches a valley from which it cannot escape using only Swap moves.

6.4 VND

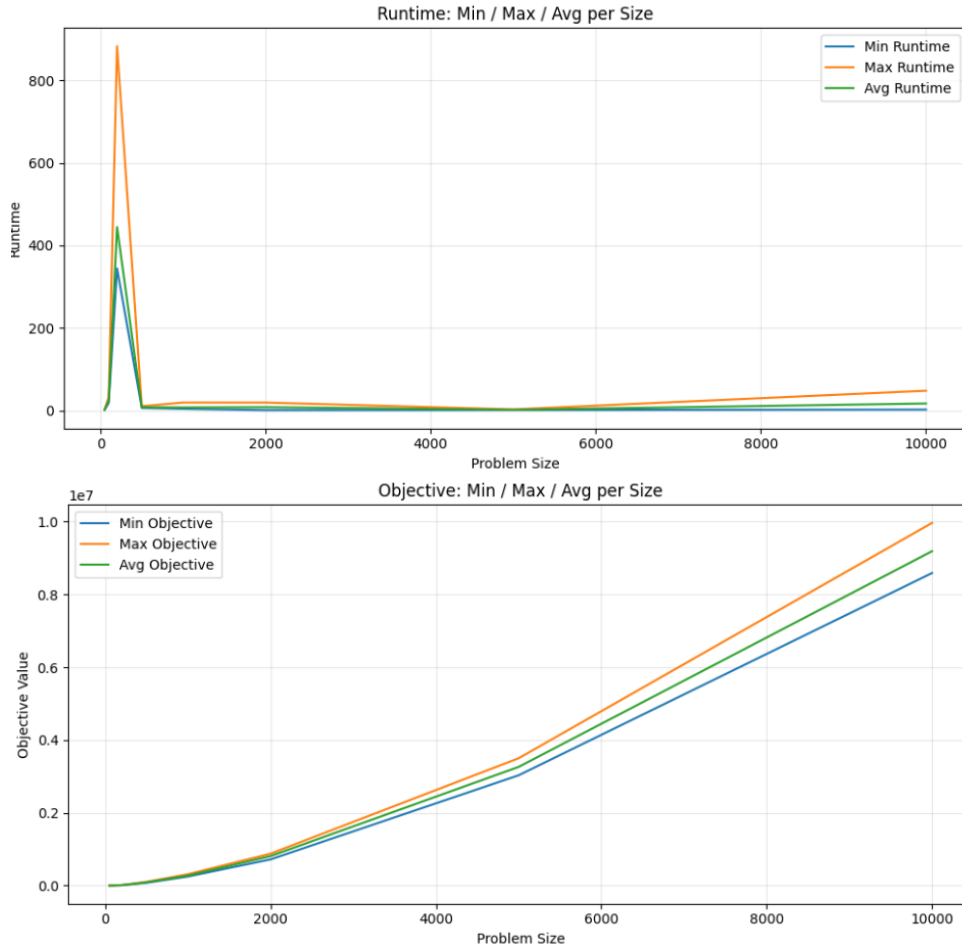


Figure 10: Runtime statistics (top) and objective function statistics (bottom): comparison of min/max/avg metrics.

Table 8: Performance summary of VND Standard Best by instance size

Size	Instances	Avg. Objective	Min Runtime	Max Runtime	Avg. Runtime
50	30	1,566.07	1.050	2.694	1.736
100	30	3,993.71	18.286	32.518	24.228
200	30	10,601.42	344.106	882.738	444.433
500	30	90,539.09	5.858	9.946	7.460
1,000	30	274,238.70	3.741	18.957	6.996
2,000	30	810,260.70	0.472	18.916	7.682
5,000	30	3,260,603.00	0.881	2.328	1.444
10,000	30	9,187,792.00	1.934	47.732	16.642

The performance analysis of the VND algorithm, utilizing the *Standard Best* configuration ($N_1 \rightarrow N_2 \rightarrow N_3$ with Best Improvement), demonstrates its capability to balance deep search exploration with runtime constraints. The average runtime drops dramatically from size 200 to size 500 and remains very low even for size 10,000 (16.64 seconds average). This

confirms that limiting the neighborhood search in large instances is essential for scalability. A notable runtime spike occurs at $N = 200$, where the algorithm records its highest average time (444.43 seconds). This represents the upper limit of the "full exploration" strategy. At this size, the search space is large enough to be computationally expensive for a *Best Improvement* strategy (which evaluates all neighbors) but small enough that our adaptive sampling had not yet been triggered. Except for that point, the runtime variance is low, suggesting predictable performance for both small and very large datasets. The objective function statistics show a consistent linear growth relative to the instance size. It closely mirrors that of the simple Local Search. This similarity suggests that the quality of the initial Greedy solution is the dominant factor in the final outcome for these instances.

6.5 GRASP

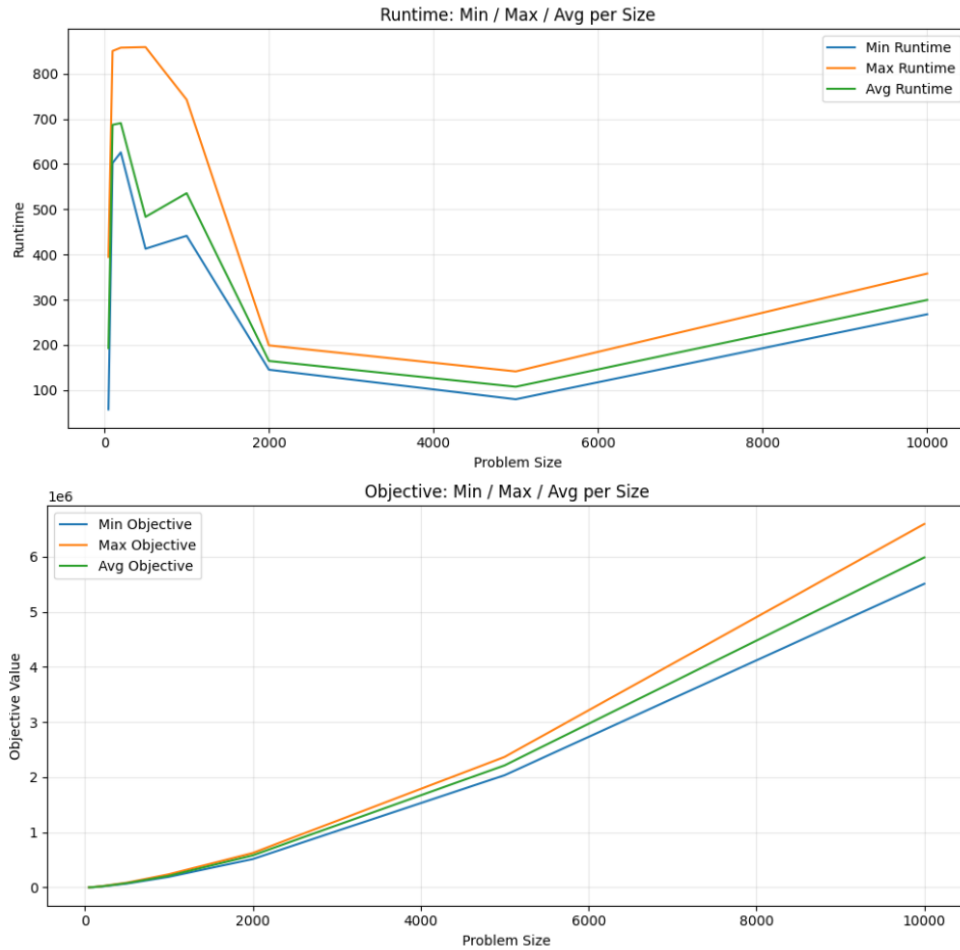


Figure 11: Runtime statistics (top) and objective function statistics (bottom): comparison of min/max/avg metrics.

Table 9: Performance summary based on aggregated statistics by instance size

Size	Instances	Avg. Objective	Min Runtime	Max Runtime	Avg. Runtime
50	30	1,568.84	57.229	394.936	192.907
100	30	5,066.91	602.426	850.500	687.078
200	30	18,947.10	626.144	857.803	690.907
500	30	77,780.75	412.751	859.093	483.356
1,000	30	209,800.70	441.582	742.695	535.770
2,000	30	582,400.00	145.153	198.955	164.446
5,000	30	2,214,979.00	79.769	141.339	107.380
10,000	30	5,989,435.00	267.832	357.822	299.393

The runtime trajectory shows a peak at $N = 1,000$ followed by a drop at $N = 2,000$. This behavior is due to two combined factors. Due to time constraints, the experiments were split between two machines. Instances $N \geq 2,000$ were executed on a computer with a faster processor. Also, the implementation reduces the number of neighbors and iterations checked as size increases. The peak at $N = 1,000$ represents the maximum load before these stricter sampling parameters take full effect. Despite these runtime optimizations, the objective value maintains a stable linear growth, confirming that the strategy effectively guides the search without the excessive computational cost of a complete neighborhood evaluation. We can check GRASP performs better than VND or simple Local Search.

6.6 Tabu Search

Performance comparison between best-improvement and first-improvement strategies. The analysis reveals a clear trade-off: best-improvement consistently achieves superior solution quality with 0.1-4.3% better objective values across all problem sizes, but at significantly higher computational cost, being 2-85x slower than first-improvement. First-improvement offers substantially faster runtime while maintaining reasonable solution quality, making it preferable for time-constrained applications, whereas best-improvement is ideal for quality-critical scenarios where computational resources are available.

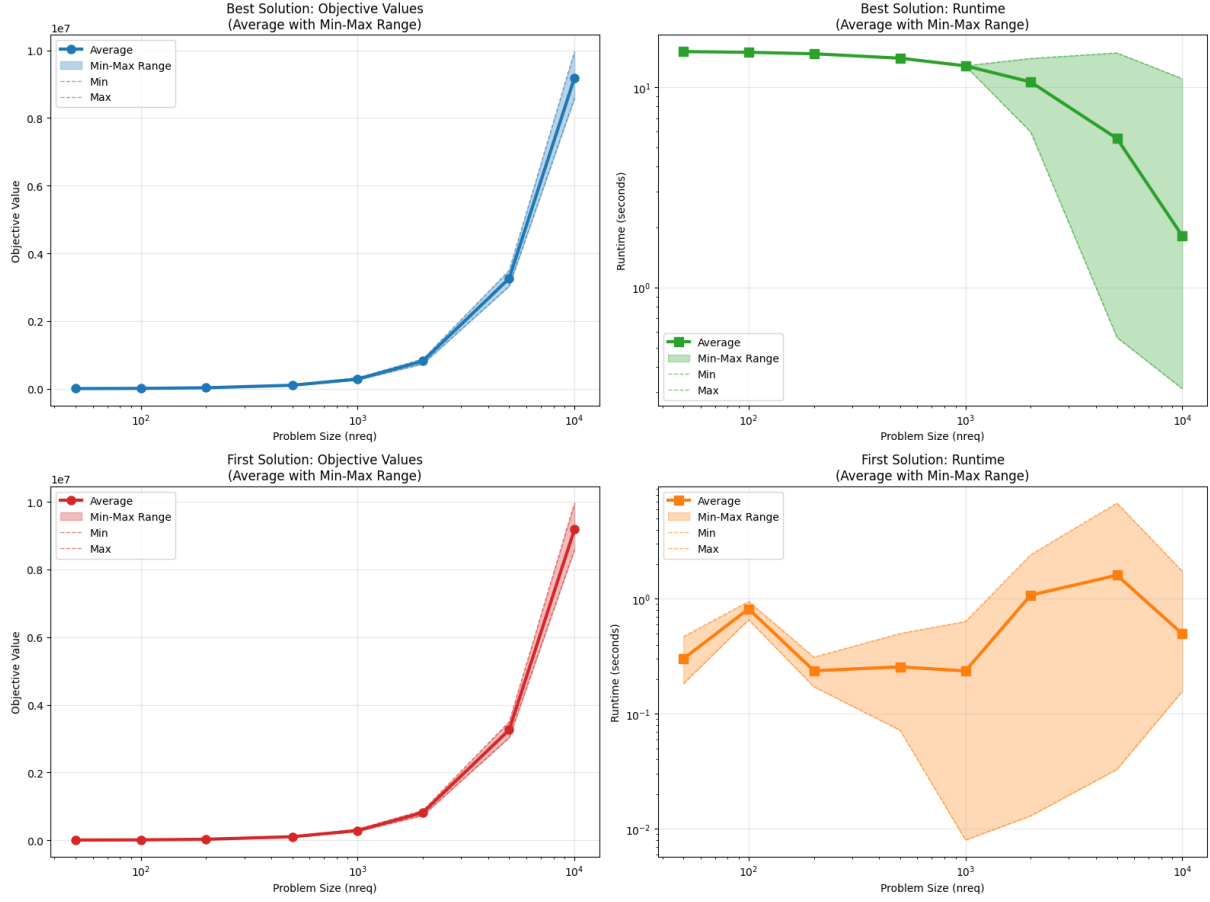


Figure 12: Performance comparison between best-improvement and first-improvement strategies across different problem sizes. Top left: Best solution objective values showing average (solid line) with min-max range (shaded area). Top right: Best solution runtime on logarithmic scale. Bottom left: First solution objective values. Bottom right: First solution runtime on logarithmic scale. The plots demonstrate the trade-offs between solution quality and computational efficiency for both approaches, with variability across instances indicated by the transparent ranges.

6.7 Beam Search

Table 10: Beam Search Algorithm Performance Across Problem Sizes

Problem Size	Beam Width	Avg. Objective	Avg. Runtime (s)	Min Runtime (s)	Max Runtime (s)
50	10	2,781.45	0.010	0.010	0.010
100	10	7,770.28	0.010	0.010	0.010
200	10	21,087.42	0.026	0.026	0.026
500	10	74,374.65	0.163	0.162	0.163
1,000	10	201,372.84	0.653	0.646	0.653
2,000	5	556,896.52	1.592	1.573	1.592
5,000	2	2,708,477.33	9.966	9.881	9.966
10,000	1	7,555,277.18	39.762	14.622	39.762

6.8 Best Solutions

To characterize the scalability of solution quality we isolated the single best result for each problem size. From the set of 30 instances executed for each size category ($N \in \{50, \dots, 10,000\}$), we selected the specific instance that yielded the global minimum objective function value using each of the algorithms.

This analysis allows us to observe the relationship between the problem dimension and the best achievable solution cost found by our heuristic.

6.8.1 Greedy Construction Heuristic

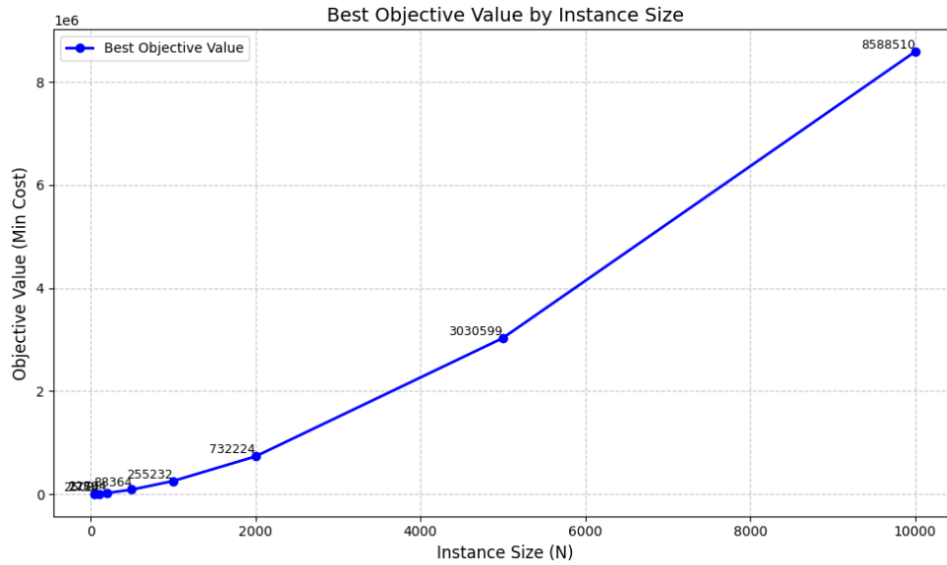


Table 11: Best performing instance per size (Minimum Objective Value).

Size	Instance	Objective	Runtime [s]
50	instance27_nreq50_nveh2_gamma41	2,507.79	0.000
100	instance20_nreq100_nveh2_gamma86	7,754.14	0.000
200	instance23_nreq200_nveh4_gamma172	22,794.04	0.000
500	instance25_nreq500_nveh10_gamma424	88,363.55	0.000
1,000	instance12_nreq1000_nveh20_gamma835	255,231.68	0.000
2,000	instance6_nreq2000_nveh40_gamma1663	732,223.61	0.001
5,000	instance28_nreq5000_nveh100_gamma4299	3,030,599.08	0.005
10,000	instance2_nreq10000_nveh200_gamma8517	8,588,510.03	1.093

6.8.2 Randomized Greedy Construction

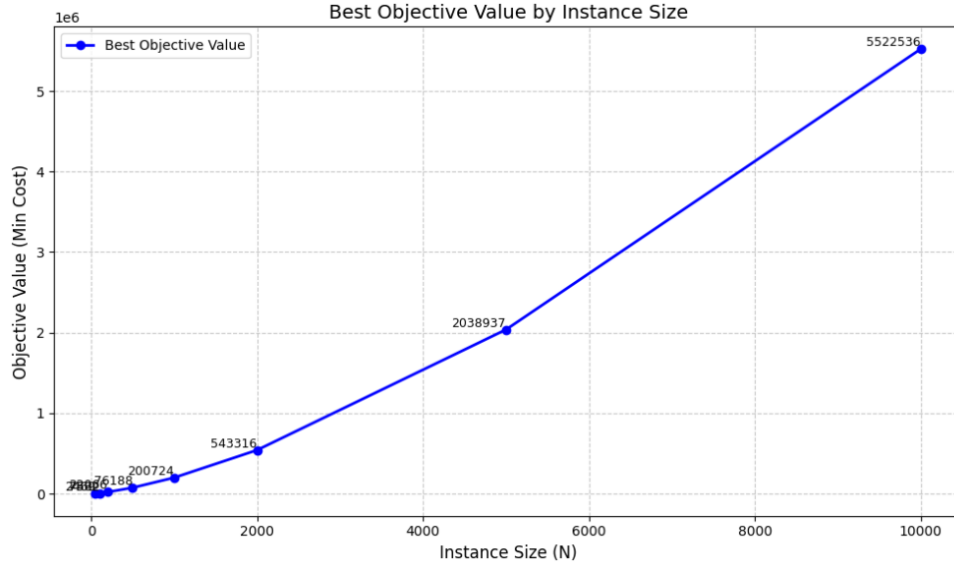


Table 12: Best performing instance per size (Minimum Objective Value).

Size	Instance	Objective	Runtime [s]
50	instance27_nreq50_nveh2_gamma41	2,469.18	0.000
100	instance20_nreq100_nveh2_gamma86	7,581.70	0.000
200	instance25_nreq200_nveh4_gamma172	22,065.75	0.000
500	instance25_nreq500_nveh10_gamma424	76,187.84	0.000
1,000	instance12_nreq1000_nveh20_gamma835	200,724.11	0.001
2,000	instance6_nreq2000_nveh40_gamma1663	543,315.71	0.002
5,000	instance28_nreq5000_nveh100_gamma4299	2,038,936.92	0.010
10,000	instance2_nreq10000_nveh200_gamma8517	5,522,536.36	0.094

6.8.3 Beam Search

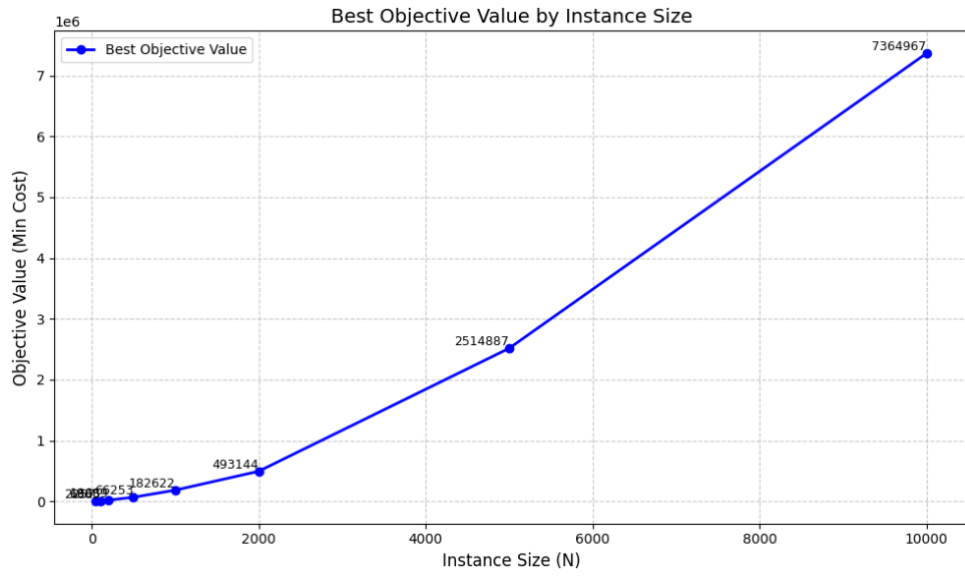


Table 13: Best performing instance per size (Minimum Objective Value).

Size	Instance	Objective	Runtime [s]
50	instance27_nreq50_nveh2_gamma41	2,257.13	0.010
100	instance7_nreq100_nveh2_gamma85	6,885.00	0.010
200	instance23_nreq200_nveh4_gamma172	18,689.20	0.026
500	instance21_nreq500_nveh10_gamma427	66,253.49	0.162
1,000	instance12_nreq1000_nveh20_gamma835	182,621.62	0.646
2,000	instance6_nreq2000_nveh40_gamma1663	493,143.96	1.573
5,000	instance28_nreq5000_nveh100_gamma4299	2,514,887.30	9.881
10,000	instance10_nreq10000_nveh200_gamma8729	7,364,967.03	39.622

6.8.4 Local Search

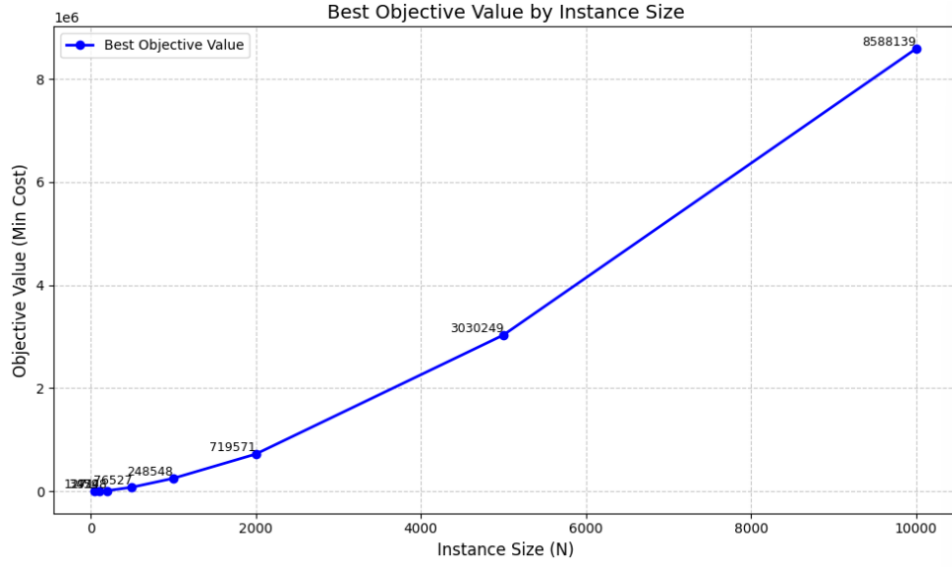


Table 14: Best performing instance per size

Size	Instance	Objective	Runtime [s]
50	instance27_nreq50_nveh2_gamma41	1,868.79	0.007
100	instance24_nreq100_nveh2_gamma84	5,582.65	0.061
200	instance23_nreq200_nveh4_gamma172	16,854.04	0.790
500	instance25_nreq500_nveh10_gamma424	65,135.11	1.015
1,000	instance12_nreq1000_nveh20_gamma835	222,731.50	31.723
2,000	instance6_nreq2000_nveh40_gamma1663	695,847.13	60.389
5,000	instance28_nreq5000_nveh100_gamma4299	3,006,637.98	60.281
10,000	instance2_nreq10000_nveh200_gamma8517	8,571,959.50	65.158

6.8.5 VND

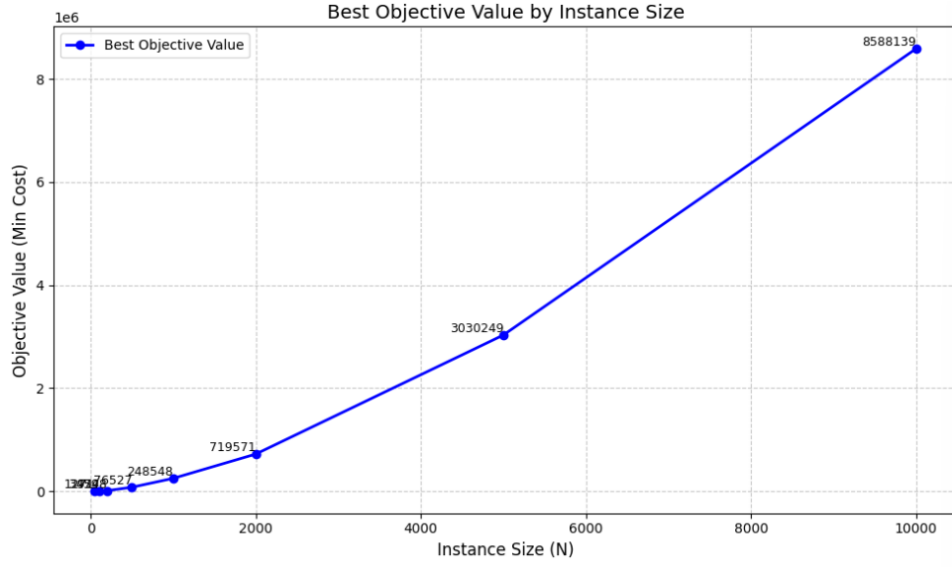


Table 15: Best performing instance per size

Size	Instance	Objective	Runtime [s]
50	instance27_nreq50_nveh2_gamma41	1,270.79	1.316
100	instance7_nreq100_nveh2_gamma85	3,459.03	24.080
200	instance28_nreq200_nveh4_gamma176	9,747.83	407.094
500	instance25_nreq500_nveh10_gamma424	76,526.55	7.753
1,000	instance12_nreq1000_nveh20_gamma835	248,547.59	4.611
2,000	instance6_nreq2000_nveh40_gamma1663	719,570.71	15.447
5,000	instance28_nreq5000_nveh100_gamma4299	3,030,249.08	1.272
10,000	instance2_nreq10000_nveh200_gamma8517	8,588,139.03	34.264

6.8.6 GRASP

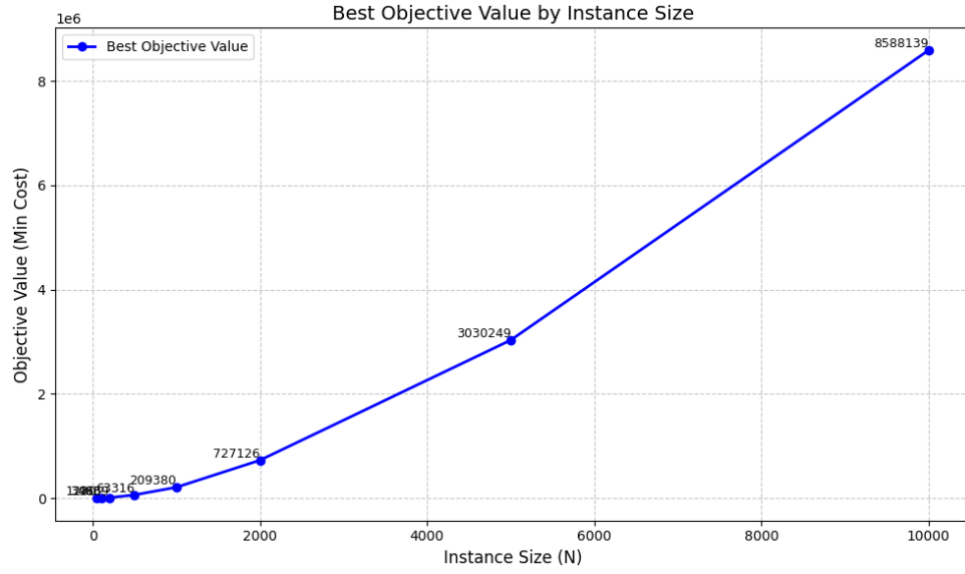


Table 16: Best performing instance per size

Size	Instance	Objective	Runtime [s]
50	instance27_nreq50_nveh2_gamma41	1,229.79	2.866
100	instance24_nreq100_nveh2_gamma84	3,482.66	22.223
200	instance28_nreq200_nveh4_gamma176	9,668.83	545.237
500	instance25_nreq500_nveh10_gamma424	63,315.55	1.467
1,000	instance10_nreq1000_nveh20_gamma870	209,379.66	11.491
2,000	instance6_nreq2000_nveh40_gamma1663	727,125.89	3.039
5,000	instance28_nreq5000_nveh100_gamma4299	3,030,249.08	0.916
10,000	instance2_nreq10000_nveh200_gamma8517	8,588,139.03	33.017

6.8.7 Tabu Search

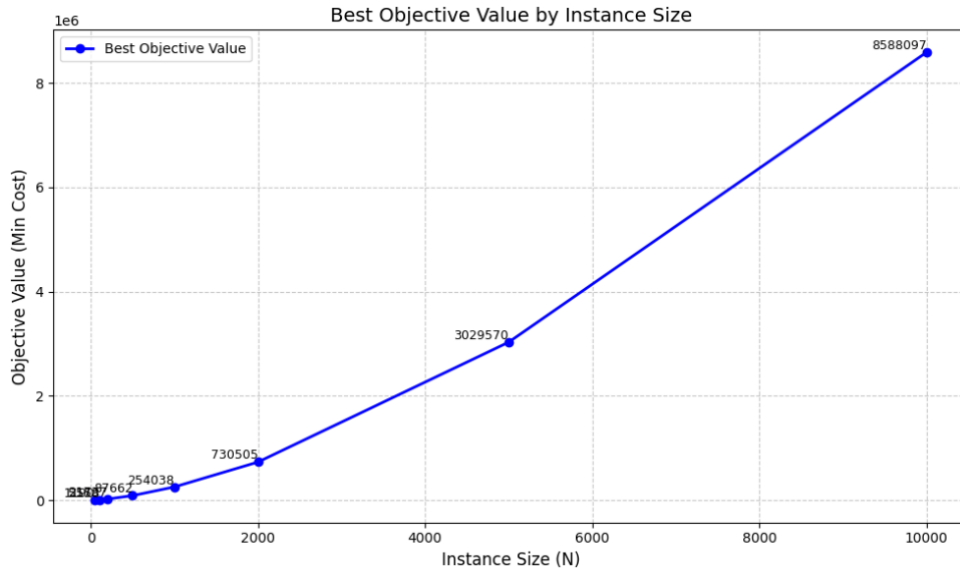


Table 17: Best performing instance per size (Minimum Objective Value).

Size	Instance	Objective	Runtime [s]
50	instance27_nreq50_nveh2_gamma41	1,189.79	15.053
100	instance24_nreq100_nveh2_gamma84	5,557.65	14.927
200	instance23_nreq200_nveh4_gamma172	21,737.04	14.679
500	instance25_nreq500_nveh10_gamma424	87,661.55	13.945
1,000	instance12_nreq1000_nveh20_gamma835	254,037.68	12.767
2,000	instance6_nreq2000_nveh40_gamma1663	730,504.61	13.350
5,000	instance28_nreq5000_nveh100_gamma4299	3,029,570.08	4.909
10,000	instance2_nreq10000_nveh200_gamma8517	8,588,097.03	0.407

6.9 Cold Start Problem

The Deterministic Greedy Construction suffers from a significant "Cold Start" bias regarding request distribution:

The algorithm aggressively assigns the "best" requests (those closest to the depot or to each other) to the first few vehicles. This results in an uneven distribution where the first routes are highly efficient and compact, while the last few active vehicles are forced to serve the remaining "leftover" requests, which are typically spatially dispersed and expensive. As a consequence the initial solution usually has a high fairness penalty (ρ) due to this disparity. This structural bias explains why pure Intra-Route optimization (N_1) is sometimes insufficient on its own for large instances, and why mechanisms like Randomized Construction (GRASP) or Inter-Route moves (N_2) are necessary to redistribute the workload more evenly.

6.10 Sensitivity to Problem Parameters

The characteristics of the final solutions are governed by the objective function

$$f(S) = \sum_{k \in K} D_k + \rho \cdot \Omega(S),$$

where ρ and γ play distinct roles.

6.10.1 Impact of Fairness Weight (ρ)

The parameter ρ regulates the trade-off between efficiency ($\sum D_k$) and equity ($\Omega(S)$).

- **Low ρ :** The problem approximates a standard Selective PDP. The solver prioritizes minimizing total travel time, often resulting in compact but unequal routes. Here, **Neighborhood 1 (Intra-Route)** is most effective.
- **High ρ :** The penalty for imbalance dominates. The algorithm may accept moves that increase total travel duration if they reduce the gap between driver workloads. **Neighborhood 2 (Inter-Route)** becomes critical here to shift requests from overloaded to underutilized vehicles.

6.10.2 Impact of Service Quota (γ)

This parameter dictates the tightness of the selection constraint.

- **Low γ :** Allows the algorithm to “cherry-pick” clustered requests near the depot, simplifying routing.
- **High γ :** Forces the inclusion of distant outliers to meet the quota. Since our Local Search does not dynamically swap served/unserved requests, a high γ places the burden of quality on the Construction Phase and GRASP to initially select a spatially compatible subset of customers.

6.11 Computational Efficiency

- **Delta Evaluation Mechanism.** The efficiency of our implementation relies heavily on two main strategies: incremental delta evaluation and adaptive neighborhood sampling, which allow the algorithms to scale to instances of up to 10,000 requests. A major bottleneck in the fairness objective function is the calculation of variance, which typically requires iterating over all routes ($O(M)$). To overcome this in the constructive heuristics (`det_cons_heu.py` and `rand_cons_heu_opt.py`), we implemented an $O(1)$ delta evaluation mechanism. By maintaining running sums of the route durations ($\sum d_k$) and squared durations ($\sum d_k^2$), the algorithm calculates the impact of an insertion without re-evaluating the entire solution. For the Local Search, although route feasibility checks remain linear with respect to the route length, the objective function update is localized to the modified routes, avoiding a global recalculation.
- **Adaptive Neighborhood Sampling.** As analyzing the full neighborhood becomes computationally prohibitive for large instances, we introduced an adaptive

sampling strategy in the VND, GRASP and TS frameworks. The algorithms dynamically limit the search space based on instance size. This transforms the complexity of a search step from quadratic to constant time relative to the total number of requests, ensuring that the solver adheres to the strict time limits while still finding high-quality improvements.

- **Runtime Bottlenecks.** Despite these optimizations, the primary remaining bottleneck is the overhead of Python’s object manipulation. Creating deep copies of Solution objects (`solution.copy()`) during the exploration of neighbors incurs a significant memory and time cost. While delta evaluation minimizes the need for full copies during the checking phase, the sheer volume of moves in the late stages of the search makes memory management the limiting factor for raw speed.